

Request scoped GitHub access

GitHub repo access is a request type **inside iam-jit** — right next to AWS IAM roles, in the same request flow. You ask for the repos and the access level; an approver reviews it; you receive a **scoped, ≤1 hour** token. This is the requester's view.

Why. A broad, standing GitHub token once let a malicious skill infect every PR of every repo it could reach. iam-jit replaces that with a token scoped to the **repos you name × one access level × ≤1 hour**, revocable at any moment — so an infected machine or agent has a far smaller blast radius.

Part 1 — Ask in the browser · **Part 2** — Ask as an agent (MCP / API) · **Part 3** — The access levels

Part 1 — Ask in the browser

The same + New request flow you already use for AWS access — just pick GitHub.

1 + New request → pick GitHub repo access

https://iam-jit.your-company.dev/requests/new

+ New request

Pick what you need. Both run the same review & approval flow.

AWS IAM role access
Paste a policy; get a scoped IAM role.

GitHub repo access ✓
Pick repos + access level; get a scoped ≤1h token.

2 Name the repos and the access level

https://iam-jit.your-company.dev/requests/new/github

Request GitHub access

An approver reviews before any token is minted; nothing is issued to repos you don't list.

organization / owner

acme

repositories (names only)

web api

access level

- ☐ Read — clone/read code, read PRs & issues
- ☒ Pull requests — open/comment/review PRs (no code push)
- ☐ Issues — open/comment issues
- ☐ Write — push code + open PRs

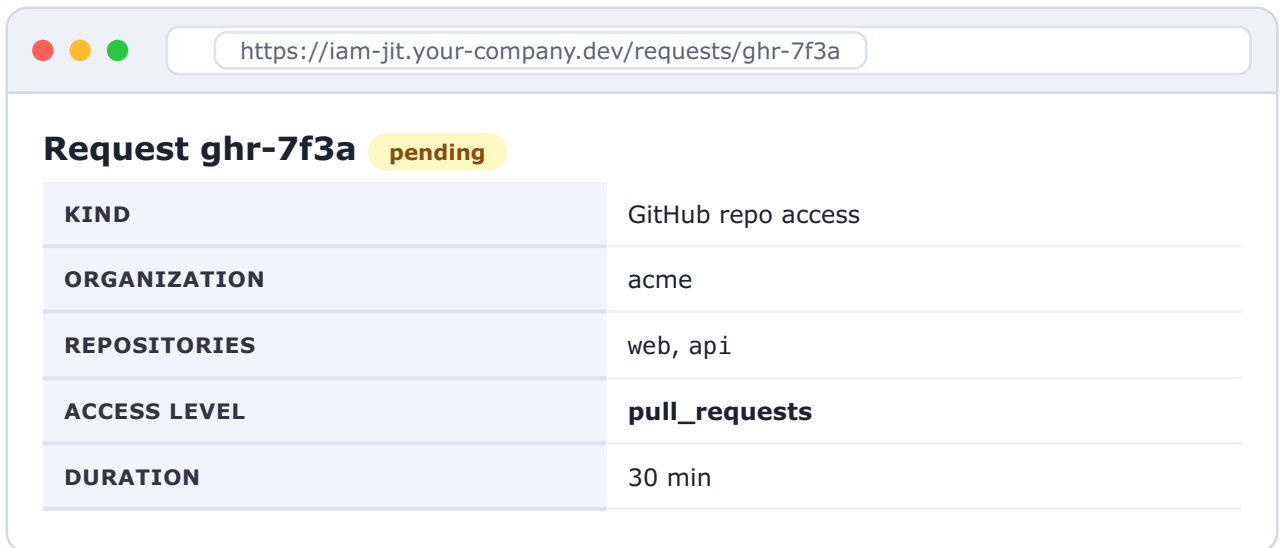
duration (minutes, max 60)

30

Submit request

3 It lands in the approver's queue

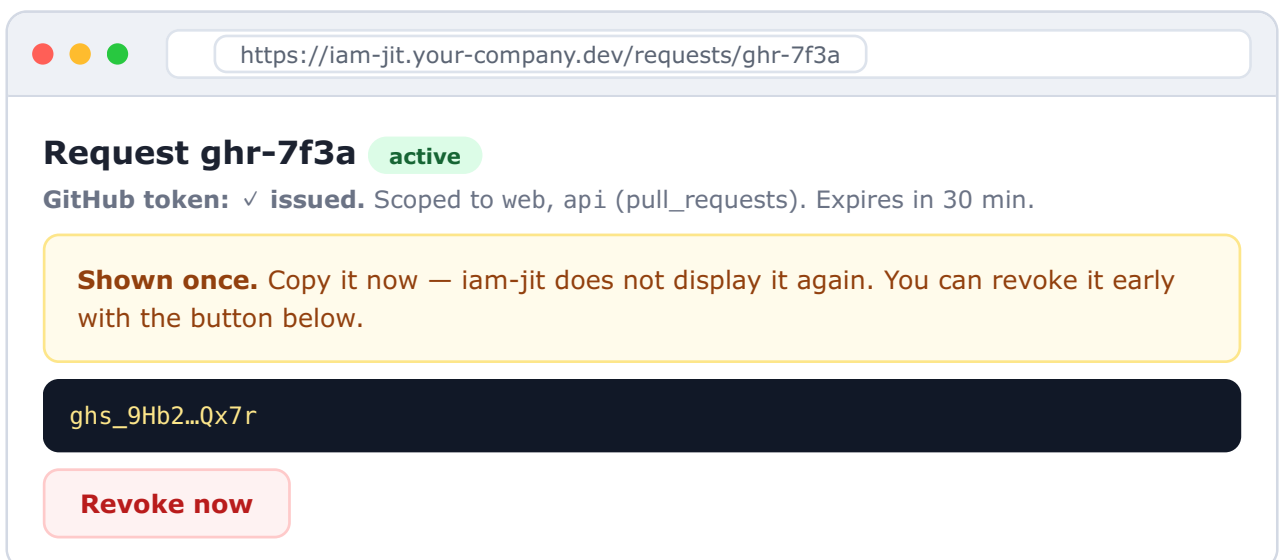
No token yet — the request waits for a human. You can watch its status on the request page.



A screenshot of a web browser window with the address bar showing `https://iam-jit.your-company.dev/requests/ghr-7f3a`. The page title is "Request ghr-7f3a" with a yellow "pending" status tag. Below the title is a table with the following details:

KIND	GitHub repo access
ORGANIZATION	acme
REPOSITORIES	web, api
ACCESS LEVEL	pull_requests
DURATION	30 min

4 Approved → your token, shown once



A screenshot of a web browser window with the address bar showing `https://iam-jit.your-company.dev/requests/ghr-7f3a`. The page title is "Request ghr-7f3a" with a green "active" status tag. Below the title, it says "GitHub token: ✓ issued. Scoped to web, api (pull_requests). Expires in 30 min." A yellow box contains the text: "Shown once. Copy it now — iam-jit does not display it again. You can revoke it early with the button below." Below this is a dark blue box containing the token `ghs_9Hb2...Qx7r`. At the bottom is a red button labeled "Revoke now".

Part 2 — Ask as an agent (MCP / API)

An agent submits the *same* request without a browser — over MCP or the JSON API — and retrieves the token once it's approved. Same queue, same approval, same $\leq 1\text{h}$ scope.

Submit the request

```
// MCP tool (or POST /api/v1/github/requests)
github_scope_self_for_task({
  org: "acme",
  repositories: ["web", "api"],
  access: "pull_requests",
  duration_minutes: 30,
  description: "open a PR fixing the build"
})
// -> { request_id: "ghr-7f3a", state: "pending" } (no token yet)
```

Poll until approved, then receive the token once

```
// GET /api/v1/github/requests/ghr-7f3a (after the human approves)
{
  state: "active",
  repositories: ["web", "api"],
  access: "pull_requests",
  expires_at: "2026-06-06T15:34:00Z",
  token: "ghs_9Hb2...Qx7r" // returned once
}
```

Use it — and GitHub enforces the scope server-side

```
// token scoped to: web, api / pull_requests
→ open a PR on web ..... 200 OK ✓ in scope
→ push code to web (not granted) ..... 403 ✗ denied
→ read repo billing (not listed) ..... 404 ✗ invisible
→ any call after 30 min / revoke ..... 401 ✗ dead
```

A fully compromised agent can only do what the grant allows — the repos you named, one access level, for under an hour, revocable at any moment. No standing, broad token to steal.

Part 3 — The access levels

Each level maps directly to GitHub functionality — pick the least that does your task.

LEVEL	WHAT IT LETS YOU DO	APPROVAL
read	clone/read code, read PRs & issues	may auto-approve
pull_requests	open / comment / review pull requests (no code push)	needs a human (or prior history)
issues	open / comment issues	needs a human (or prior history)
write	push code + open pull requests	needs a human (or prior history)

Reading code is the only level that can be auto-approved. Anything that can **modify** — PRs, issues, or code — is reviewed by a person, unless an admin has recorded a prior approval for you on that repository. Either way the token you receive is always freshly minted and ≤ 1 hour — there is never a long-lived token sitting around.

ALWAYS TRUE OF A GITHUB GRANT

Scoped to exactly the repositories you named — others return 404.

Expires automatically in ≤ 1 hour (GitHub's hard ceiling).

Revocable instantly, before expiry, from the request page.

The token is shown to you exactly once; iam-jit never stores it in any list.